



INFT-4041 · BSIT · 5TH SEMESTER · FALL 2026
E-Commerce Application Development

APPENDIX

C

Answers to the Checkpoints

Taught in

—

Laboratory

—

Assessment

—

This is a class extract. It is *Appendix C* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall12026_Handout.pdf`.

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — What E-Commerce Is, and What It Changed

1. **Electronic business, not electronic commerce.** Viewing results and downloading transcripts are business processes conducted over a network, but no value changes hands through the system — the transaction happens at a bank counter. The moment fee payment moves online it becomes e-commerce, and the university acquires every problem to the right of the dashed line in Figure 1.1: a payment that may have succeeded while the network dropped, a receipt that must exist exactly once, a refund path.
2. Many are acceptable. One good answer: **refusing to sell the last item when it is visibly damaged.** In software this requires a stock record that distinguishes “held” from “sellable”, somebody whose job is to update it, a rule for what the storefront does when the two disagree, and a decision about what the customer sees — out of stock, or a quiet substitution? The shopkeeper does all of that in half a second without recording that a decision was made, which is exactly why it is forgotten when the shop goes online.
3. **Two uncounted costs:** the acquisition cost of finding customers who currently arrive free (Chapter 2 puts this at Rs. 1,200 an order for a typical small shop, which can exceed the commission), and the labour of running the four functions themselves — payment integration, delivery arrangements, customer service, returns. The commission was buying those, not just the listing. **What they gain that is not money:** the customer relationship. They learn who bought, can contact them again, and cannot be delisted. For a business with repeat custom that is frequently worth more than the 15%.

Chapter 2 — Business Models and Revenue Models

1. **Business model:** a marketplace connecting restaurants with diners, with fulfilment performed in-house — three and a half of the four functions. **Revenue models: three at once** — *commission* from restaurants, a *service or delivery fee* from customers, and *advertising* in the form of promoted placement. **Most profitable per unit: the advertising.** It has almost no marginal cost, where commission is earned against riders and support that must be paid for, and the delivery fee rarely covers the rider. Promoted placement is close to pure margin, which is why these platforms push it so hard.
2. Viable when $LTV > CAC$, that is when $400 \times n > 1,500$, so when the average customer orders $n > 3.75$ — **at least four times** — before they stop. *What to measure:* the repeat rate and the time over which it happens. Four orders over six months is a business; four orders over six years is not, because the cash was spent today and returns over a period during which the venture has to survive.
3. B2C platforms store **one price per product** — a column on the product row. B2B pricing is a *relation* between customer and product, often with quantity breaks and expiry dates. That is not a setting you have failed to find; it is a different cardinality, and making it work means either a different platform or an extension that intercepts every price lookup in the system

(Chapter ?? shows how much work that is). The assumption that breaks is “a product has a price”.

Chapter 3 — Auctions, Portals and Communities

1. **Bid Rs. 15,000** — your true valuation. You win, and you pay the second-highest bid, **Rs. 13,500**, gaining Rs. 1,500. Bidding Rs. 18,000 does not improve this outcome at all: you still pay Rs. 13,500, because the price is set by the others. What it does is expose you to winning when somebody bids Rs. 16,500, at which point you pay Rs. 16,500 for a thing worth Rs. 15,000 to you. The overbid can only ever buy you an auction you did not want.
2. It **never closes**. The auction stays open indefinitely; the winning bidder is never notified, the seller is never paid, and the listing accumulates further bids whenever somebody wanders past. The seller sees an item that appears to be permanently “ending soon” and a buyer who has given up. Worse, the failure is *silent* and inversely proportional to interest: popular auctions close correctly and unpopular ones do not, so the bug is invisible in testing, where you load every page you create.
3. Any two of: **be a seller yourself** until real ones arrive (cost: you take on stock and inventory risk, and you compete with the sellers you are recruiting); **narrow the market** to one city or one category (cost: you cap your growth and may pick the wrong niche); **give sellers a tool worth having without buyers** (cost: you are now building and supporting a second product); **list supply gathered elsewhere** (cost: legally risky and reputationally expensive if discovered).

Chapter 4 — Planning an E-Commerce Framework

1. (a) **Functional**, and it *does not* discriminate — all four options reset passwords. (b) **Functional**, and it **discriminates sharply**: per-customer pricing eliminates most hosted platforms and the marketplace route outright, and is the single most common reason to choose self-hosted or bespoke. (c) **Non-functional**, and it discriminates *in the opposite direction* to what students expect — surviving a twentyfold spike is something a large hosted platform does effortlessly and your single virtual server does not.
2. **Hosting** (Rs. 324,000 in the worked example), **setup and theming labour** (Rs. 175,000 plus Rs. 100,000 of custom modules), and above all **maintenance labour** (Rs. 360,000). Their stated saving of Rs. 792,000 is therefore closer to Rs. 339,000 — still a saving, but less than half what they claimed, and it vanishes entirely if maintenance takes eight hours a month instead of four.
3. Many are acceptable. A clear one: **a home baker selling six kinds of cake to one city, taking orders through Instagram**. Self-hosting would mean paying for a server and maintaining it to serve perhaps thirty orders a week, with nobody to patch it — which is not a cheaper shop but an unpatched one. **Choose marketplace-only or a cheap hosted platform**: the business’s scarce resource is the baker’s time, the volume is low, nothing about the pricing is unusual, and every hour spent on a server is an hour not spent baking.

Chapter 5 — The Stack, and Your Shop in One Command

1. **Working**: the operating system and the web server — a 500 is generated and served by Apache, so the request reached it and a response came back. **Failed**: the application layer; PHP raised an error and died before producing a page. The database is *unknown* — it may be the cause (the application could not connect) or entirely healthy. **Look first** at the PHP error log, with `docker compose logs shop`; the message is never on the screen, because `display_errors` is off, and Chapter ?? is about why you want it that way.

2. **depends_on** orders container *starts*, and a container that has started is not a database that will accept a login — MariaDB opens its TCP port several seconds before it finishes initialising. The shop therefore starts, runs the installer, and the installer is refused by a database that is listening but not ready. **The failure is intermittent and worse on a fast machine**, because the faster the shop container starts, the more certainly it arrives early. The **service_healthy** condition waits for the database’s own healthcheck to pass, which tests a login rather than a port.
3. **Lost:** both named volumes — the entire database (products, categories, orders, customers, settings) and the shop’s installed files, including any module you had written in place. **Not lost:** anything in the repository — the Compose file, the Dockerfile, `code/`, and any module kept there rather than edited inside the container. **What would have made it minor:** a database dump taken on a schedule and kept outside the volume, plus the discipline of writing modules in `code/modules/` and mounting them, so that the only thing destroyed is state you can regenerate. Chapter ?? makes both of these routine.

Chapter 6 — Products, Categories and Their Tables

1. **Move to oc_product_description:** the product name and the meta description. Both are *words*, and words differ by language. **Stay in oc_product:** price, weight and stock quantity — a kilogram is a kilogram in Urdu, and a shop that let the price depend on the reader’s language would eventually charge two different prices for one product. The test is the first of the four questions: *does this fact depend on who is reading?*
2. **Buys:** a single-query answer to “every category beneath this one”, so a category page costs one query instead of a loop whose length depends on how deep the tree happens to be. **Costs:** the same fact is now stored twice, so the two can disagree, and every write that changes the shape of the tree must rewrite the derived rows. **The operation that makes it visible:** *moving a category*. Changing one **parent_id** invalidates the path rows of that category and of everything beneath it, so the admin panel must rewrite them all — which is why moving a category with many descendants is noticeably slower than renaming one.
3. Two of several good answers. (a) **The database was never the bottleneck:** if the database is 20% of the page, halving it improves the page by 10%, which can be smaller than the run-to-run variance. (b) **The expensive thing is the number of queries, not the cost of any one** — the measurement in this chapter found 146 queries for one page, and an index helps exactly one of them. **To tell them apart:** turn on the query log with `long_query_time=0`, request the page once, and read `COUNT(*)` and `SUM(query_time)` from `mysql.slow_log`. A large total with few queries is (a); a small total spread over a hundred queries is (b). That single measurement separates them, and it is the one in `m06_catalogue.py`.

Chapter 7 — Product Variations, Options and Uploads

1. **The photograph is an option** — specifically a **file** option. It is supplied by the customer, it is different every time, and the shop holds no stock of it. **The size is a variant**, because a small mug and a large mug are two different objects on two different shelves, each with its own cost, its own barcode and its own count. **What the shop holds in stock:** blank mugs in two sizes. Nothing else — which is exactly why the photograph cannot be a variant: there is no such thing as stock of “the mug with Ahmed’s photograph on it” until Ahmed orders one.
2. **Option value rows:** $4 + 3 + 2 = 9$, plus three rows in `oc_product_option`. **As variants:** $4 \times 3 \times 2 = 24$ combinations, so **25 oc_product rows** — twenty-four plus the master — and 25 description rows, 25 store rows and 25 category rows beside them. Nine against a hundred, for the same catalogue.

3. **Passes:** the filename sanitiser (nothing illegal in “logo.svg”), the length check, the extension allow-list (svg is on the stock list), and the MIME allow-list — it claimed `image/png`, which is allowed, and nothing verifies that claim. **Fails:** nothing. All five checks pass. **For it to harm a visitor**, the file must be *served to them from your origin and rendered*. Two things would have to be true: the attacker must learn the stored URL, which carries a 32-character random token and so is not guessable — but it *is* shown to whoever uploaded it and may be forwarded to an administrator who opens it — and the server must serve it with `Content-Type: image/svg+xml` rather than as a download, which Chapter 5’s stock install does. The realistic victim is therefore the administrator reviewing the order, and the realistic fix is to drop `svg` from the list and serve uploads with `Content-Disposition: attachment` (Chapter ??).

Chapter 8 — Themes, Layout and the Customer’s Experience

1. **Best case for compressing the photographs:** the images are $713 - 597 - 54 =$ about **61 KB**. Halving them saves 30 KB, which is 4% of the page — and that is the *best* case, because you cannot compress them to nothing. **Propose instead:** minify `bootstrap.css`, which is 265 KB of the 597 and has no minified copy in the tree. That single change saves roughly 240 KB, about 34% of the page — eight times the whole image budget — and changes nothing a customer can see. Then remove the unused parts of Bootstrap and FontAwesome, and only then look at the images.
2. **Layout change** if a “trust badge” module already exists: **Design > Layouts > Product**, add the module to **Content Bottom**, set its sort order. No code, visible to the next person, survives upgrades. **Template override** if it must sit *immediately under the button*, inside the markup the theme generates: there is no position there, so you override `product/product.twig` in your child theme and add it. The general rule: a position that exists is a layout change; markup that must differ is a template change.
3. **Two reasons it is lower.** (a) **It follows only assets named in the HTML.** Fonts are requested by the CSS once the browser parses it, so none of the FontAwesome `.woff2` files — 248 KB in that directory — are counted. (b) **It does not execute JavaScript**, so anything a script loads afterwards (a tracking pixel, a lazy-loaded image, a carousel’s later slides) is invisible to it. **Which it could fix:** the first, by parsing the CSS for `url()` references and following them — a dozen lines. The second it cannot fix, because fixing it means running a browser, which is a different kind of tool. Knowing which limitation is laziness and which is fundamental is the useful judgement here.